

Plexus

A blueprint for differentiable tissue

Janelia Research Campus

September 22, 2026

Il semble que la perfection soit atteinte non quand il n'y a plus rien à ajouter, mais quand il n'y a plus rien à retrancher.

Antoine de Saint-Exupéry, *Terre des hommes* (1939), ch. III

What I cannot create, I do not understand.

Richard P. Feynman, blackboard at Caltech, February 1988

Abstract

A mechanism, in the account that philosophers of biology have converged on, is *entities and activities organized such that they are productive of regular changes from start or set-up to finish or termination conditions*.¹ We introduce *Plexus*, a language for writing a mechanism down in exactly those terms, so that the description can be run, fitted to observations, and shown to be missing an activity. A model is stated in two halves. *What exists*: the entities, of as many kinds as the hypothesis needs, the states they carry, the relations between them and the levels that organize them. *What happens*: the activities acting on them. Neither half is subordinate to the other. An activity such as adhesion has no meaning apart from what is adhering to what, and a model of a tissue, its basement membrane and the surrounding matrix is informative precisely because those remain distinct kinds of entity rather than being flattened into variables of one universal object. Every activity therefore declares the entities it acts upon, the states it requires, the relations it needs, and one or more numerical implementations, while a hierarchy of levels keeps heterogeneous entities, fields and relations organized rather than merged. Decomposition serves recomposition: the purpose is to take existing models apart into reusable pieces and to build new biological compositions from them, which in turn requires the language that specifies a model to be separated from the interpreter that executes it. The Plexus research program has three objectives. First, to define an algebra of activities over these organized entities. Second, to decompose the existing corpus of simulation models into this common language. Third, to construct differentiable implementations of these activities so that mechanistic models can be directly inferred from biological data.

1 Introduction

Computational biology has produced a rich ecosystem of simulation frameworks. Neural simulators model electrical activity, reaction–diffusion systems model chemical transport, particle and continuum methods model mechanics, while vertex, Cellular Potts and active-matter models describe

¹Machamer, P., Darden, L. & Craver, C.F. (2000). Thinking about mechanisms. *Philosophy of Science* 67:1–25, p. 3. <https://doi.org/10.1086/392759>. The hierarchical reading is developed in Craver, C.F. (2007), *Explaining the Brain*, Oxford University Press, and Craver & Darden (2013), *In Search of Mechanisms*, University of Chicago Press.

multicellular tissues. These frameworks remain however largely incompatible. Combining electrical signalling, mechanics, metabolism and development often requires coupling multiple specialized simulators whose underlying abstractions differ fundamentally. Each simulator introduces its own representation of what a mechanism is, making models difficult to compare, compose and extend. This fragmentation is not primarily a numerical problem but a language problem.

Biology already has an account of what a mechanism is, even if it is rarely written out. In the account of Machamer, Darden and Craver, a mechanism has two halves. *Entities* are the things: they have properties and stand in spatial relations to one another. *Activities* are what the things do: they are the producers of change. And the two are *organized*. Spatially, by location, shape and contact. Temporally, by order, rate and duration. Hierarchically, because what the parts do at one level constitutes what the whole does at the next. A mechanism starts from set-up conditions and ends at termination conditions, and its description bottoms out in activities that the field takes as fundamental. This is the account a cell biologist uses tacitly whenever a paper ends with a summary figure: named parts, arrows between them, boxes inside boxes. The paper has exact words for that figure. A *mechanism sketch* is a description with gaps, black boxes that no known activity yet fills. A *mechanism schema* is an abstract description whose placeholders can all be filled with known parts and activities. A complete description exhibits *productive continuity without gaps* from set-up to termination conditions, and the continuity lies in the arrows. The sketch cannot be run, cannot be fitted to data, and cannot be shown to be missing an arrow.

Plexus takes that account literally. Instead of treating simulation frameworks as fundamental, we treat entities and activities as the primary objects, and the biological system itself as the thing to be decomposed. A system is decomposed not only into activities but also into the entities those activities act upon. These entities are heterogeneous, receptors, signalling proteins, cells, extracellular matrices, tissues, fields, and they are organized across biological levels. Plexus makes that organization explicit as a compositional hierarchy: each level carries its own entities and states, activities act between entities within a level, and Aggregate and Broadcast connect levels by mapping state upward and downward. Decomposition therefore does not flatten biology into a single collection of variables; it preserves the organization through which mechanisms at one level are composed with mechanisms at another.

On the activity side, Plexus writes each one down independently of its numerical implementation: adhesion, diffusion, division, chemotaxis, electrical propagation or active stress. In a specification an activity is written as an *operator*; the two words name the same thing, and this paper uses the biological one except where it names code. Entities and activities are the two halves of one decomposition, and neither is primary. An activity is typed by the entities, states and relations it requires, and a population of entities is inert until activities act on it.

The purpose of this decomposition is ultimately recomposition: to construct complete mechanistic models from reusable entities and activities, rather than merely to classify the mechanisms of existing simulators. Making such compositions executable requires a strong structural choice. The language that specifies a biological model must be separated from the code that interprets and executes it. The language declares entities, states, relations, hierarchy and activities; the interpreter resolves that description into concrete numerical implementations and schedules their execution. In this perspective, biological modelling becomes analogous to compiler design: just as modern compilers separate programming languages from their implementations through an intermediate representation, Plexus separates biological mechanisms and their organization from the numerical methods that realize them. The analogy is a consequence of wanting free recomposition, not the premise it starts from. What the language guarantees is that a hypothesis is well formed and

complete. Whether it is *productive of* the change observed is what running it establishes.

This viewpoint defines a broader research program. First, Plexus seeks to construct an algebra of activities whose elements can be composed into complex dynamical systems, compared through their observable consequences, and expanded as new mechanisms are discovered. Second, Plexus aims to extract this algebra from the existing corpus of biological simulation models. Rather than viewing current frameworks as competing simulators, we regard them as repositories of reusable entities and activities. By decomposing these models into a common language, we seek to identify a compact vocabulary capable of expressing a wide range of biological dynamics across scales. Third, Plexus constructs differentiable counterparts of these compositions. Differentiability allows mechanistic models to be fitted directly to biological data, residuals to be interpreted mechanistically, and missing activities to be intuited.

Finally, because mechanisms are represented in a formal language rather than embedded in simulator-specific code, they become amenable to automated reasoning. The explicit separation between biological semantics and numerical implementations allows agentic systems to operate directly on the language itself, composing, comparing and optimizing mechanistic models without modifying simulator code. The search for biological explanations therefore becomes a search over model structures rather than simulator implementations.

2 The language

Plexus defines a typed language for expressing biological models independently of their numerical realization. The language names *what exists*: the *entities*, the *states* they carry, the *fields* they live in, the *relations* between them and the *hierarchy* that organizes them. And it names *what happens*: the *activities* that transform them. Biological models are therefore represented as compositions of activities acting on organized discrete and continuous biological objects rather than as simulator-specific algorithms. The following subsections take these in turn: activities, entities, states, fields, and the hierarchy that holds them.

2.1 Activities

Activities are the fundamental objects of the language. An activity represents a single biological mechanism, such as adhesion, chemotaxis, diffusion, electrical propagation, active stress, metabolism, growth, or cell division. Every activity is defined by a typed signature and one or more numerical implementations. The signature specifies *what* biological transformation is performed, whereas implementations specify *how* it is numerically realized. The signature specifies

- the entities the activity acts on,
- the entities it changes,
- the state variables consumed and produced.

Activities are therefore typed morphisms between kinds of entities. What an activity does *not* carry is the relation it acts through. Maps define the relations a mechanism requires — which particle belongs to which cell, which synapses connect which neurons, how cells relate to a continuous field — and each is declared once, with the structure that carries it: the containment map π of a set, or the two legs of a relation given entity status (Section 2.5). An activity reaches the relations it needs by traversing those declarations at run time, through the gather and scatter primitives the language provides, rather than by restating them in its own signature.

The separation is what makes activities reusable across organizations. One adhesion acts over a neighbour graph, a mesh half-edge table or a contact list without being rewritten, and a Rewire that changes the relation changes which entities participate without touching the activity at all. It also keeps the two declarations from drifting apart, which they do as soon as both exist: a signature that names the maps it walks is a second copy of a fact the structure already states, and the copy is only as good as the discipline maintaining it.

2.2 Entities

Entities come in kinds: genes, proteins, cells, neurons, synapses, tissues, organs and organisms. The entities of one kind form a set S , homogeneous in kind: its elements share one state schema and one biological interpretation. In what follows a *kind* of entity and the set that holds it are the same object seen from biology and from mathematics.

Kinds organize into hierarchies reflecting biological organization, but a level of that hierarchy is *not* a single kind, and this is the point at which a familiar simplification has to be resisted. Writing the hierarchy as a chain $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$, molecules, then cells, then tissues, then organism, suggests one biological category per level. Biology is not so uniform. A cell contains receptors, signalling proteins, cytoskeletal elements and, in a mechanical model, material points; a tissue contains cells, a basement membrane and an extracellular matrix. Each of those is a distinct kind with its own state and its own activities, and all of them share a parent. A level is therefore a heterogeneous collection

$$L_k = \{ S_k^1, S_k^2, \dots, F_k^1, F_k^2, \dots, R_k^1, R_k^2, \dots \},$$

of kinds S_k^i , fields F_k^j bound to that level, and relations R_k^l among them. The hierarchy organizes these objects; it does not define them by a single molecule/cell/tissue label. Each level is expressed in the same language, allowing intracellular signalling, multicellular mechanics, developmental processes and organ-level physiology to be described without translation between formalisms.

2.3 States

Every entity carries a state describing its current biological condition. While entities define *what* exists, states define *what each entity carries*. The state schema is specific to each kind. A protein may carry position, binding state and the load it bears; a material point position, velocity and deformation gradient; a neuron membrane voltage and calcium; a cell polarity, stiffness and gene-expression levels; a synapse conductance and plastic weight. Activities transform states. They consume selected state variables and produce updates to others, while leaving the underlying entities unchanged.

2.4 Fields

Fields represent continuous quantities defined over space rather than over discrete entities. Examples include morphogen concentrations, pressure, extracellular signalling molecules, electric potentials, and material properties. Fields complement entities. While entities are the discrete parts, fields describe the continuous environment, the microenvironment, in which these parts evolve. Activities couple entities and fields through explicit maps relating discrete entities to continuous spatial representations.

2.5 Hierarchy

Containment is declared, not implied. Entities of one kind may be contained in entities of another, proteins in a cell, cells in a tissue, tissues in an organism, through an explicit map π from children to parent. A parent may contain several distinct kinds at once: the same cell can hold receptors, signalling proteins, cytoskeletal elements and material points, each remaining a distinct object with its own state and its own activities while sharing that parent. Every level carries its own state, its own activities and its own rate, so no level is reduced to a parameter of the level above it.

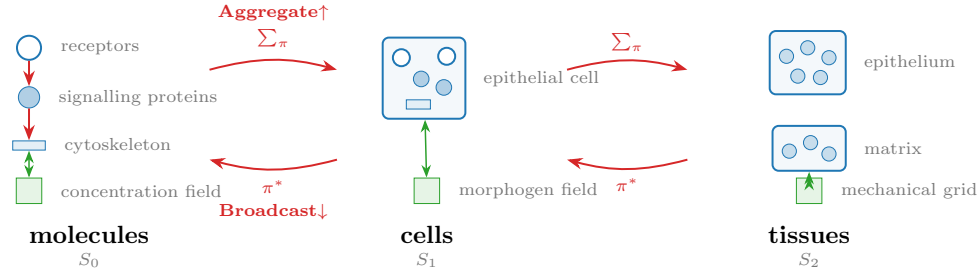


Figure 1: The hierarchy. Each level holds several *distinct* kinds of entities (blue) and may carry its own fields (green); activities (red) act between entities within a level, and exactly two families cross a level: Aggregate, $\sum \pi$, and Broadcast, π^* .

Activities act within a level or across levels. Within a cell, activities may couple receptors to signalling proteins to cytoskeleton. Across the containment map, exactly two families apply. **Aggregate** is integration: it builds parent state from children, a cell position from its material points, a neuron input from its synapses. **Broadcast** is top-down control: it propagates parent state to children, a material property to every point a cell owns, or a constraint imposed by the parent. Formally,

$$L_k \xrightarrow{\text{Aggregate}} L_{k+1}, \quad L_{k+1} \xrightarrow{\text{Broadcast}} L_k.$$

π is a *function*, and that is load-bearing rather than a formality. A function partitions the children into fibres $\pi^{-1}(p)$, and it is that partition which makes Aggregate well defined: every child is counted exactly once, so the sum conserves. Broadcast is its adjoint. A many-to-many relation would admit neither. Aggregating over one needs weights to avoid counting a shared child several times, and no weighting is canonical.

Many-to-many relations need no new primitive. Much biological structure is not a function. A vertex of an epithelial mesh belongs to three cells at once, so vertex $\rightarrow$ cell cannot be written as π . This does not call for a second kind of hierarchy: a relation $R \subseteq A \times B$ is a kind of entity R together with two functions $R \rightarrow A$ and $R \rightarrow B$, so a relation is represented by giving it the status of entities, and both of its legs are ordinary containment maps. Synapses with **pre** and **post** are exactly this, a many-to-many relation between neurons carried as a population of pairs, and so is the half-edge table of a mesh, whose every half-edge has exactly one source vertex and exactly one face. Aggregating half-edge quantities onto faces (a cell's area, its perimeter) is $\sum \pi$ along the second leg, and reading vertex positions at half-edges is π^* along the first. The two cross-level families are therefore sufficient for structure that is not itself hierarchical, provided the relation is given the status of entities.

Fields are level-specific in the same way. Exchange couples entities to a field at their own level, so a molecular concentration field, a cellular morphogen field and a tissue-scale mechanical grid coexist as distinct objects rather than being merged into one grid.

The consequence is that molecules, cells and tissues are not a pipeline of separate simulators handing results to one another. They are one compositional graph: several kinds may coexist inside each parent, activities connect them locally, and Aggregate and Broadcast connect the levels. The same families are reused throughout, so adding a biological scale adds no machinery to the engine.

3 The algebra of activities

The language introduced in the previous section defines the objects manipulated by Plexus. We now define the elementary families of activities from which biological models are constructed. The central hypothesis of Plexus is that a large fraction of existing biological simulation models can be expressed as compositions of a small number of elementary families. Rather than introducing a new activity for every biological phenomenon, Plexus identifies a compact algebra that can be reused across mechanics, signalling, metabolism, development and physiology.

The proposed algebra consists of eight elementary families (Fig. 2), each named here first in a biologist’s words:

- **Lateral:** what neighbours do to each other among entities of one kind.
- **Aggregate:** integration, a whole reading its parts, many-to-one across levels.
- **Broadcast:** top-down control, a whole imposing on its parts, one-to-many across levels.
- **Exchange:** traffic between entities and their microenvironment, the fields.
- **Rewire:** remodelling, a change of the relations between entities.
- **Divide:** proliferation, the creation of entities.
- **Die:** apoptosis and extrusion, the removal of entities.
- **Seed:** the set-up conditions, the establishment of the initial state.

The first four families transform biological state while preserving the underlying organization. Rewire modifies the relations, whereas Divide and Die modify the entities themselves. Seed writes the state a model starts from. Together they form the elementary computational vocabulary of Plexus.

Seed is separated from Divide and Die, with which it shares a mechanism since all three write state directly rather than returning a delta, because it differs in *when*. Divide and Die act throughout a trajectory; Seed acts only at its opening. Treating that distinction as a naming convention rather than a kind has a concrete cost. In an automated model-search campaign on a growing epithelial vesicle, an initial condition for a reaction–diffusion field was re-applied on every timestep rather than once, which converts it from an initial condition into a moving boundary condition and silently overwrites the state of any activity writing to the same channel. Both spellings were legal specifications and nothing in the language could distinguish them; the resulting runs reported a mechanism as refuted eight times across thirteen rounds when the activity under test had in fact never reached the state. Because Seed is a kind, the runtime imposes its window, and a per-timestep initial condition is not expressible.

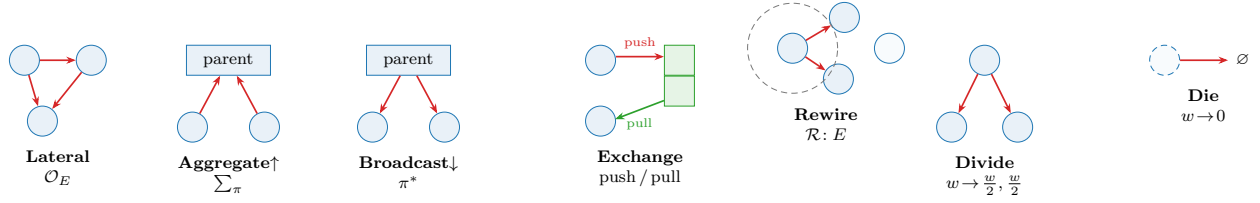


Figure 2: The elementary algebra of activities of Plexus. Lateral, Aggregate, Broadcast and Exchange transform biological state. Rewire modifies relations, whereas Divide and Die modify the entities. Complex biological models are obtained by composing these elementary families.

Lateral. Lateral activities are what entities of one kind do to one another. Typical examples include mechanical forces between proteins, adhesion and tension between neighbouring cells, juxtacrine signalling, electrical communication between neurons, or biochemical interactions within a metabolic network.

Aggregate. Aggregate activities are integration: a whole reads its parts, propagating information from many entities to a higher level of organization. Examples include computing cellular properties from their constituent proteins or accumulating synaptic currents acting on a neuron. This is the upward direction of causation, the one usually called emergence.

Broadcast. Broadcast activities are top-down control: a whole imposes on its parts, propagating information from higher organizational levels toward their constituents. Aggregate and Broadcast therefore define the upward and downward flow across biological levels.

Exchange. Exchange activities couple discrete entities with continuous fields: a cell's traffic with its microenvironment. Typical examples include secretion, sensing, transport, uptake, or particle-grid transfer.

Rewire. Rewire activities are remodelling. They modify the relations between entities without changing the entities themselves, updating neighbour graphs, contact networks, developmental connectivity or other interaction structures, and so determining which entities participate in subsequent activities.

Divide and Die. Living systems continuously modify their own composition; this is turnover. Divide creates new entities, whereas Die removes existing ones. These activities modify the biological organization itself rather than only its state, and they are what makes a tissue different from any inert material.

4 Composition

A biological model is built by composition, and composition happens in four distinct ways. Each answers a different question, and a model is complete only when all four are answered.

1. **Entities compose into structures.** The kinds of entities, the states they carry and the relations declared between them form the objects a model is about: a membrane is a population of faces with an incidence relation, a connectome is a population of neurons with an edge

population over it, a tissue is a population of cells with a containment map to its molecules. This composition is structural and is fixed by the specification before any dynamics run.

2. **Activities compose within a level.** Several activities may act on the same entities, each contributing its own increment: adhesion and active stress on one population of cells, diffusion and reaction on one chemical field. Because activities are typed by what they read and write rather than by the simulator they came from, mechanisms from different modelling traditions may act side by side on the same entities.
3. **Levels compose through Aggregate and Broadcast.** A mechanism written at one level reaches another only through the containment map of Section 2.5, $L_k \rightarrow L_{k+1}$ upward and $L_{k+1} \rightarrow L_k$ downward. This is what allows a molecular mechanism and a tissue-scale mechanism to belong to one model without either being rewritten in the other’s terms.
4. **Activities compose in time.** The three compositions above say what acts on what; the *schedule* says in which order and how often. This is the temporal organization of the mechanism: its order, rate and duration.

The first three are the subject of the preceding sections. The remainder of this section formalizes the fourth.

Rather than embedding all mechanisms within a monolithic simulator, Plexus represents a model as an ordered sequence of activities. This sequence, called the *schedule*, specifies both the order and the frequency with which activities are evaluated. Formally, one simulation step is the composition

$$\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1,$$

whose repeated application generates the system dynamics

$$x_{t+1} = \Phi(x_t).$$

The schedule therefore describes the temporal organization of biological mechanisms. Different activities may evolve on different time scales. Mechanical interactions may be evaluated every simulation step, whereas diffusion, growth or cell division may occur less frequently. A biological model therefore becomes a multi-rate composition of biological mechanisms rather than a monolithic numerical solver.

Activities are pure transformations. Rather than modifying biological state directly, every activity returns its contribution to the system evolution. If activity $\mathcal{O}_i$ produces the increment Δ_i , the complete update is obtained by

$$\Delta = \sum_i \Delta_i,$$

before being passed to the numerical implementation. This formulation has two important consequences. First, biological mechanisms remain independent and composable: introducing a new mechanism simply requires adding a new activity to the schedule. Second, the language remains independent of its numerical realization. The schedule specifies the biological model, whereas the numerical implementation specifies how that model is evaluated.

5 Numerical implementations

Every mechanistic description stops somewhere. It bottoms out in activities the field is content to take as given. In Plexus that floor is the numerical implementation. The algebra of activities defines the biological semantics of a model independently of its numerical realization. Every activity may admit multiple implementations satisfying the same typed signature. These implementations differ in numerical realization while preserving identical biological semantics. Selecting an implementation is therefore a numerical decision rather than a biological one.

This abstraction naturally accommodates a broad range of computational methods. Mechanical activities may be implemented using Material Point Methods, finite elements, vertex models, phase-field methods, Cellular Potts models or optimal-transport formulations. Signalling activities may employ Hodgkin–Huxley equations, graph neural networks or Neural ODEs. Likewise, field activities may use finite differences, finite elements, neural implicit representations or spectral methods. These implementations coexist within the same language because they satisfy identical contracts.

Different implementations may also employ different numerical integration schemes. Depending on the activity, implementations may evolve first-order states such as concentrations or membrane voltages, second-order states such as mechanical accelerations, or rely on explicit, implicit or adaptive solvers. These choices affect computational efficiency and numerical accuracy but not the biological interpretation of the model.

This distinction fundamentally changes how biological simulations are compared. Traditional frameworks compare complete simulators, intertwining biological mechanisms with numerical algorithms. Plexus instead separates these two questions. Biological models are compared through the activities they compose, whereas numerical methods are compared through the implementations realizing those activities.

Because biological mechanisms are represented in a formal language rather than embedded in simulator-specific code, they become amenable to automated reasoning. The explicit separation between biological semantics and numerical implementations allows agentic systems to operate directly on the language itself, composing, comparing and optimizing mechanistic models without modifying simulator code. The search for biological explanations therefore becomes a search over model structures rather than simulator implementations.

6 Mechanistic inverse modelling

One of the primary motivations for introducing a formal language of biological mechanisms is to identify mechanistic models directly from experimental observations. Rather than calibrating simulations manually, the objective is to infer the parameters, spatial organization and constitutive laws governing biological systems from imaging data.

Plexus makes this possible by associating every activity with one or more differentiable implementations satisfying the same typed signature. Differentiability therefore becomes a property of the language rather than of a particular simulator. Regardless of the numerical backend, gradients propagate through compositions of explicit activities rather than through monolithic simulation code.

This distinction fundamentally changes the role of inverse modelling. Traditional approaches assume that the underlying mechanism is already correct and seek the parameter values that best

reproduce the observations. Such parameter optimization is effective when the forward model is complete, but it becomes misleading when important activities are absent. Optimizing existing parameters may improve the fit while merely compensating for missing activities.

In Plexus, parameters remain attached to explicit activities rather than to opaque numerical models. Mechanical properties belong to mechanical activities, reaction rates to biochemical activities, conductances to signalling activities, and spatial fields to the mechanisms generating them. Parameter estimation therefore preserves the mechanistic interpretation of the model.

Residuals likewise acquire a mechanistic interpretation. A persistent mismatch between simulation and experiment no longer indicates only poor parameter values; it may also indicate that the current composition of activities is unable to explain the observations. In the terms of Machamer, Darden and Craver, a residual that will not go away is the mark of a sketch: a black box where an activity belongs. Differentiability thus provides more than efficient optimization: it enables quantitative comparison between competing mechanistic models while preserving their explicit biological structure.

Crucially, because the composition is differentiable end to end, the objective need not span the entire system: a loss may be *local* in both space and time. It can be evaluated on a chosen *subset of entities*, a region of interest such as the extrusion front where activated and quiescent cells meet, over a *short rollout* of a few tens of steps, rather than on every cell across the whole trajectory. Gradients still propagate through the shared composition, so a *local* objective fine-tunes precisely the mechanism responsible for a local behaviour, the shape of a growth front, the sharpness of a domain boundary, the timing of a rearrangement, without the averaging-out of a global fit and, decisively, without the prohibitive cost of differentiating a long, whole-system rollout. This last point is what makes the approach practical at all: a loss spanning every cell over hundreds of frames unrolls, under reverse-mode differentiation, into a backpropagation graph whose memory and compute scale as cells $\times$ steps and quickly become intractable, whereas a loss confined to a few entities over a short window keeps that gradient graph small. *Locality* is therefore the enabling property of differentiable refinement on large, long-lived tissues: the mechanism is global and shared, but the *evidence* constraining any single activity is usually confined to a small, transient region, and the loss should be placed exactly there.

7 Agentic mechanistic discovery

“What I cannot create, I do not understand.”

“Know how to solve every problem that has been solved.”

Richard P. Feynman²

Feynman’s two lines frame this section. A biological mechanism is understood only when an agent can *create* it, assembling it from activities until it reproduces the observations, and new discovery begins only once the agent can already re-derive every model that has been solved. The three loops below make this operational, moving from reconstructing what is already known toward constructing the mechanisms that existing models cannot yet express.

The explicit representation of biological mechanisms transforms scientific discovery from a param-

²Found written on Feynman’s blackboard at Caltech at his death in February 1988. Never published in a paper or lecture, it survives as a bare aphorism; the second line stood directly beneath the first on the same board. Photographs of the blackboard are widely reproduced.

eter optimization problem into a language exploration problem. Because activities are declared rather than embedded in simulator-specific code, they become directly accessible to agentic systems.

Plexus proposes a three-loop strategy for mechanistic discovery. Each loop operates at a different level of abstraction, progressively moving from existing models toward new biological knowledge.

Loop I: Understanding. The first loop takes an existing model, a paper and its repository, and decomposes it into Plexus activities, entities, states and fields, so the mechanisms it implements become explicit. That decomposition is necessary, but it is only the *entry*: it makes the model’s *levers* explicit without yet explaining them. Understanding is earned afterwards, by *interrogating* the decomposition as a causal system, learning what each activity and parameter does on its own and, the harder part, what it does in combination, since mixtures rarely surrender their causal structure to inspection. The driving force is therefore not translation but a *hypothesis* $\rightarrow$ *validate/-falsify* cycle run on the decomposed model: each step states a falsifiable claim about a lever, tests it against a dedicated metric, and accumulates the causal map that results. A useful operating point is a rough **70/30 balance of validation to falsification**. Pure validation only ever confirms hypotheses one already believes (trivial, near-zero information), while pure falsification only ever probes the improbable (which mostly breaks as expected and never consolidates a working map), so most tests should confirm plausible levers while a deliberate minority attacks the current understanding, where a surprise is most informative. The loop’s product is a causal lever-map of the model, what each mechanism buys and how mechanisms combine, which is what makes the activities genuinely *reusable* rather than merely catalogued, and what later tells Loop III which lever the model *lacks*.

Loop II: Fitting. The second loop estimates the parameters, spatial fields and set-up conditions of the resulting mechanistic model by fitting it directly to experimental observations. Because every activity admits a differentiable implementation, optimization preserves an explicit correspondence between parameters and biological mechanisms.

Loop III: Discovery. The third loop analyses the residual remaining after optimization. Persistent discrepancies are interpreted as evidence that the current composition of activities is incomplete rather than simply poorly parameterized. Turning Loop I’s hypothesis-driven validate/falsify cycle outward, from the levers the model *has* to the ones it *lacks*, the agent proposes candidate activities, compares competing mechanistic hypotheses, and extends the algebra whenever existing mechanisms fail to explain the observations. It turns a sketch into a schema, one filled box at a time.

Together, these three loops transform simulation, inverse modelling and hypothesis generation into successive stages of a single scientific workflow. The language itself becomes the object of discovery: new biological knowledge is represented by extending the algebra of activities rather than merely adjusting parameter values.

Orchestration: loop agents and a master controller. The three loops are realized as autonomous agents coordinated by a *master*: each performs one kind of work, Understanding, Fitting, or Discovery, and communicates only through typed artifacts on a shared blackboard (the mechanistic model, the success metrics with current values, the residual, and a queue of falsifiable hypotheses). The master never performs the loops’ work; it is a state machine whose guards are

the metrics. It advances a stage when its metric is met, invokes Discovery when a residual persists, and re-invokes Fitting once a new activity is admitted. Beyond scheduling it carries a few responsibilities: it *authors* the success metrics, so “goal achieved” is a falsifiable test rather than a visual impression; it *grounds* hypotheses in the reference literature and code before Discovery proposes; it gates every proposal on *compliance* (a well-typed Plexus contract, not a simulator-specific patch) and *consistency* (an active search for violated invariants, determinism, conservation, agreement between overlapping runs, that separates “the model is incomplete” from “the measurement is wrong”); and it holds each stage open until settled, refusing the “good enough to move on” shortcut.

The master reproduces a modeller’s discipline. These responsibilities are not invented for the machine; they formalise how a careful modeller works, distilled from a prototype that deciphers the *tyssue* epithelial vertex model into Plexus activities.³ Four disciplines recur.

- **Verify the instrument before trusting the measurement.** The sharpest, most-repeated lesson: a “97% hollow” shell that was a recording-stride *misalignment* (vertex positions paired with another frame’s connectivity, caught by a determinism contradiction, a long run diverging from its own shorter prefix), and a false “0 invalid faces” that passed self-intersecting bow-tie polygons because the guard checked area > 0 , not a simple embedded tiling. An agent that skips this gate elaborates activities to explain an artefact.
- **Small, falsifiable, metric-gated steps: one factor at a time, cheap-first.** Loop I’s validate/falsify cadence, enforced across every stage: isolating proliferation rate, division timing and volume stiffness is what homogenised the tissue; a blind sweep would not attribute the win.
- **Perfecting.** A satisfied metric is not qualitative fidelity: the triangular-versus-round-cells gap became a newly authored metric (shape index, sliver fraction) and a target to close.
- **A persistent, revisable master memory.** A log of hypotheses validated and overturned, grounding reads from the reference literature, and the atlas of activities, so the search accumulates knowledge instead of re-deriving it each round.

What the loop cannot yet supply, and the agents that must. A prototype run with a human in the loop makes the division of labour precise, and honest about its frontier: the worker agents are strong at execution, breadth and bookkeeping, but four contributions came reliably from the human and name what the scheme must still internalise.

- **Goal custody.** The single most valuable intervention. A worker elaborates whatever it is touching and cannot catch its own drift (“I thought we were starting tubing from 2000 cells”); the master’s objective must be held *separately* from the workers’.
- **Temporal perception.** A genuinely structural gap: an agent reads a static strip but cannot *watch a movie*, yet the decisive evidence was in the time course (a pattern that forms only at 12s; a spot that decays then reforms; tip cells that balloon at 2s). A dynamics-watching critic is a required agent, not a convenience.
- **Domain priors and taste.** Reading the activator gradient off Okuda’s figure; naming amplification feedback before finding it is Gierer–Meinhardt.

³Reported separately as a living document of dated field notes: it reconstructs the model as mechanism-named contracts and works each morphogenesis stage by the method below.

- **Literature grounding.** “which reaction–diffusion regime does Okuda use?” → the GM unlock.

A faithful multi-agent scheme is therefore not one master and identical workers but a small division of labour, a goal-custody supervisor, a dynamics-watching critic, and a literature-grounding reader, around the disciplined loops above.

A Reference corpus

The atlas of activities is built by decomposing open-source models for which both the publication and the code are public. The corpus, by theme.

Multicellular and tissue mechanics. [MathCancer/PhysiCell](#), [PhysiBoSS/PhysiBoSS](#), [MathCancer/BioFVM](#), [Chaste/Chaste](#), [CompuCell3D/CompuCell3D](#), [Morpheus](#), [BioDynaMo/biodynamo](#), [tissue-forge/tissue-forge](#), [rmerks/VirtualLeaf2021](#), [HaseloffLab/CellModeller](#), [RobertGregg/CellularPotts.jl](#), [ingewortel/artistoo](#), [mathbioleiden/Tissue-Simulation-Toolkit](#), [shabaz/gpu-cpm](#); active vertex model, multicellular simulations with optimal transport, cell simulation as cell segmentation.

Vertex, Voronoi and epithelial mechanics. Okuda et al. 2013, 2015, 2018; [DamCB/tyssue](#), [tvm \(ZhangTao-SJTU/tvm\)](#), graph vertex model ([neoVM](#)), [alhashmiii/3D-Vertex-Model](#), [sussmanLab/cellGPU](#), [Pablo1990/pyVertexModel](#), [yasuhiroinoue/PyCellVertex](#), [mmarinriera/VertexModel](#), [mshagirov/vertex_simulation](#), [kylechanphy/TissueSim](#), [vmansov/clustering_vmodel](#), [lixinzhi4429/VM_tissue_growth](#), [BanerjeeLab/HNP_model](#); differentiable: [VertAX \(VirtualEmbryo/VertAX\)](#), [jax-morph \(fmottes/jax-morph\)](#), differentiable design for morphogenesis, mean-field moduli of a 3D vertex model; reaction–diffusion on growing domains: [Ledesma-Durán 2023, 2026](#); tubes, lumens and sheets: [Yu & Li 2024](#), [Yu et al. 2024](#), [Fujiwara et al. 2022](#), [Noguchi & Elgeti 2024](#), [Fuji et al. 2022](#), [Moore, Dale & Woolley 2023](#); surveys: [Briñas-Pascual et al. 2024](#), [Neagu et al. 2026](#), [Sorichetti et al. 2026](#), [Cook et al. 2022](#), [Courcoubetis et al. 2022](#).

Reaction–diffusion, spatial molecular and transport processes. [Morpheus](#), [readdy/readdy](#), [ssandrews/Smoldyn](#), [mcellteam/mcell](#), [mcellteam/cellblender](#), [CNS-OIST/STEPS](#), [virtualcell/vcell](#), [GollyGang/ready](#), [spatial-model-editor/spatial-model-editor](#); reaction–diffusion in intracellular transport and control, a programmable reaction–diffusion system.

Intracellular signalling and biochemical networks. [copasi/COPASI](#), [RuleWorld/bionetgen](#), [sys-bio/tellurium](#), [sys-bio/roadrunner](#), [pysb/pysb](#), [GillesPy2/GillesPy2](#), [StochSS/StochSS](#), [AMICI-dev/AMICI](#), [opencobra/cobrapy](#), [BhallaLab/moose-core](#).

Neural circuits. [jaxleyverse/jaxley](#), [brainpy/BrainPy](#), [neuronsimulator/nrn](#), [brian-team/brian2](#), [nest/nest-simulator](#), [arbor-sim/arbor](#), [AllenInstitute/bmtk](#), [Neurosim-lab/netpyne](#), [LFPy/LFPy](#), [jonescompneurolab/hnn-core](#), [neurolib-dev/neurolib](#), [NeuralEnsemble/Nengo](#).

Mechanobiology and collective behaviour. [ERK-mediated mechanochemical waves](#), [collective cell migration](#), [bioelectric morphogenesis](#).

Inverse modelling and differentiable simulation. [MultiCell](#), differentiable rendering for 3D fluorescence microscopy, inferring junction tension and pressure from cell geometry, interpretable models of cell mechanics from protein images, 3D single-cell shape analysis.

Image analysis and quantitative biology. [mpicbg-scicomp/tissue_miner](#), [charlesnchr/ERnet-v2](#), [DeXtrusion](#), [FlowShape](#), [theislabs/cellrank](#).

Interactive artificial life. [Lenia](#), [Particle Lenia](#), [Neural CA](#), [Convolutional CA](#), [Multi-neighborhood CA](#), [Game of Life](#), [Evoloops](#), [Germs](#), [Particle Life](#), [Life Engine](#), [Alien Project](#), [Bibites](#), [Synthetic Selection](#), [Subnautica](#).